Fundamentals of Pre-Silicon Validation Winter-2026

Verification of Asynchronous FIFO

VERIFICATION TEST PLAN

Team -10

Monesh Kareti - monesh@pdx.edu

Senkathir Mutharasu - senkmuth@pdx.edu

Contents

1. Introduction 3
1.1 Purpose of the Verification 3
1.2 Specifications for the Design 3
2. Verification Requirements 5
2.1 Verification Levels 5
3. Required Tools 5
3.1 List of Necessary Software and Hardware Tools 5
3.2 Directory Organization and Computer Resources 5
4. Tests and Methods 6
4.1 Testbench Architecture 7
5. Verification Strategy 7
5.1 Verification Plan 7
5.2 Edge Case Testing (Partially Implemented) 8
6. Coverage Metrics (To be Implemented) 9
7. Verification Environment 9
7.1 Class-Based Testbench Architecture 9
7.2 UVM-Based Testbench Architecture 11
8. Resources Requirements 13
Team Members and Task Allocation 13
9. References 13

1. Introduction

1.1 Purpose of the Verification

The purpose of this verification plan is to methodically assess the functionality, performance, and reliability of the asynchronous FIFO design developed for ECE-593. The key objectives include:

Functional Verification

- Ensuring the FIFO correctly follows the First-In-First-Out (FIFO) data movement principle.
- Validating the proper assertion and deassertion of the full, empty, full_3_4, and empty_1_4 status flags.
- Confirming that binary and Gray-coded read/write pointers operate correctly across all boundary conditions.

Asynchronous Operation

- Ensuring data integrity between write and read clock domains, both operating at 500 MHz but with asymmetric idle cycles.
- Verifying correct operation of the two-stage synchronizer modules (sync_wptr and sync_rptr) for safe cross-domain pointer transfer.

Metastability Condition

- Confirming that Gray-code pointer encoding in wptr_handler and rptr_handler eliminates multi-bit transitions during clock-domain crossing, mitigating metastability risks.

Cross-Clock Domain Signals

- Verifying signals that transition between clock domains: g_wptr / b_wptr synchronized into the read domain, and g_rptr / b_rptr synchronized into the write domain.
- Checking handshaking signals, FIFO status flags, and pointer consistency under concurrent access scenarios.

This verification plan aims to achieve comprehensive coverage of the asynchronous FIFO's functionality, ensuring high confidence in its accuracy, reliability, and performance across all specified usage scenarios.

1.2 Specifications for the Design

The following parameters define the design under verification, derived directly from the RTL source files (hw_top.sv, wptr_handler.sv, rptr_handler.sv):

Parameter	Value
Sender (Module A) Clock Frequency	500 MHz (wclk)
Receiver (Module B) Clock Frequency	500 MHz (rclk)
Write Idle Cycles	0 – continuous write every clock cycle

Read Idle Cycles	3 – one read every 4 clock cycles
Burst Size	450 data items
FIFO Depth (parameter DEPTH)	512 (2^9)
Pointer Width (PTR_WIDTH)	$clog2(512) = 9 bits
Data Width (DATA_WIDTH)	16 bits (parameterized)
Minimum Required FIFO Depth	360 entries
full_3_4 threshold (wptr_handler)	(b_wptr − b_rptr_sync) ≥ 384
empty_1_4 threshold (rptr_handler)	(b_wptr_sync − b_rptr) ≤ 384

Timing Calculations:

- Time to write one data item: 1 × 2 ns = 2 ns
- Time to write a full burst of 450 items: 450 × 2 ns = 900 ns
- Time to read one data item: 5 × 2 ns = 10 ns
- Total items read within 900 ns: 900 ÷ 10 = 90 items
- Data remaining in FIFO after one full burst: 450 − 90 = 360 items
- Implemented FIFO depth: 512 (2^9) — exceeds the 360-entry minimum; chosen for power-of-2 pointer arithmetic simplicity.

Important Signals and Flags

Signal	Description
wclk	Write-domain clock (500 MHz)
wrst_n	Active-low asynchronous reset for write domain
w_en	Write enable — data written when high and FIFO not full
data_in [15:0]	16-bit data input from sender
rclk	Read-domain clock (500 MHz)
rrst_n	Active-low asynchronous reset for read domain
r_en	Read enable — data read when high and FIFO not empty
data_out [15:0]	16-bit data output to receiver
full	Asserted when FIFO cannot accept more data
empty	Asserted when FIFO has no valid data
full_3_4	Asserted when occupancy ≥ 384 (3/4 of 512)
empty_1_4	Asserted when occupancy ≤ 128 entries remain

2. Verification Requirements

2.1 Verification Levels

Verification is performed at the top level using the asynchronous_fifo module (hw_top.sv) as the Device Under Test (DUT). The system hierarchy is as follows:

- System Top – instantiates the Producer (write-side stimulus), asynchronous_fifo DUT, and Consumer (read-side checker).
- asynchronous_fifo – integrates four sub-modules: synchronizer (×2), wptr_handler, rptr_handler, and fifo_mem.
- wptr_handler – manages the binary and Gray-coded write pointer; generates full and full_3_4 flags.
- rptr_handler – manages the binary and Gray-coded read pointer; generates empty and empty_1_4 flags.
- synchronizer – two-flop synchronizer for cross-clock-domain pointer transfer (instantiated as sync_wptr and sync_rptr).
- fifo_mem – synchronous-write / synchronous-read dual-port memory array of depth 512 × 16 bits.

3. Required Tools

3.1 List of Necessary Software and Hardware Tools

Software Tool Sets:

- ModelSim / QuestaSim – primary simulation, waveform analysis, and coverage collection tool

3.2 Directory Organization and Computer Resources

Source Code Files (RTL):

- hw_top.sv
- fifo_mem.sv
- wptr_handler.sv
- rptr_handler.sv
- synchronizer.sv

Testbench Files – Class-Based:

- fifo_packet.sv
- fifo_generator.sv
- fifo_driver.sv
- fifo_monitor_in.sv
- fifo_monitor_out.sv

- fifo_scoreboard.sv
- fifo_env.sv
- fifo_if.sv
- fifo_test.sv
- tb_top.sv
- fifo_pkg.sv

Testbench Files – UVM-Based:

- fifo_pkg.sv
- fifo_packet.sv
- fifo_sequencer.sv
- fifo_seqs.sv
- fifo_driver.sv
- fifo_monitor.sv
- fifo_monitor_in.sv
- fifo_subscriber.sv
- fifo_scoreboard.sv
- fifo_agent.sv
- fifo_env.sv
- fifo_tb.sv
- fifo_test_lib.sv
- fifo_if.sv
- tb_top.sv

Scripts:

- run.do – ModelSim/QuestaSim compilation and simulation automation script

4. Tests and Methods

Three testing methodologies are applied across the verification effort: Black Box, White Box, and Gray Box.

Black Box Testing:

- Verifying asynchronous operation by performing concurrent read and write actions through the DUT top-level ports only.
- Ensuring proper assertion and deassertion of the full, empty, full_3_4, and empty_1_4 flags in all defined scenarios.
- Validating data integrity — data written via data_in must appear correctly on data_out in FIFO order.
- Checking that a burst of 450 items is correctly accommodated by the 512-deep FIFO without data loss.
- Confirming correct waveform behavior at 500 MHz for both wclk and rclk domains.

White Box Testing:

- SystemVerilog Assertions (SVA) access internal signals: b_wptr, b_rptr, g_wptr, g_rptr, g_wptr_sync, g_rptr_sync.
- Pointer Gray-code consistency: g_ptr == (b_ptr >> 1) ^ b_ptr is asserted every cycle.
- Synchronizer pipeline stages g_q and b_q are observed to confirm two-cycle latency.

Gray Box Testing:

- Monitor observes internal occupancy via b_wptr and b_rptr to augment functional coverage bins.
- full_3_4 and empty_1_4 threshold logic (≥ 384 and ≤ 384 respectively) is cross-checked with pointer arithmetic in the scoreboard.

4.1 Testbench Architecture

The UVM testbench follows a layered structure. The Test layer instantiates a top-level UVM Environment (fifo_tb) which holds the Scoreboard and an inner UVM Environment (fifo_env). The inner environment contains the UVM Agent and Coverage Subscriber. The Agent encapsulates the Sequencer, Driver, and two Monitors — one per clock domain. The Scoreboard compares full packet objects observed from both monitors using a queue-based reference model. The DUT connects exclusively through the SystemVerilog interface bundle.

5. Verification Strategy

Simulation was selected for functionality testing due to the design's parameterized nature and the flexibility it provides for updating testbenches across milestones. Both directed and constrained-random test generation are employed. The UVM environment enables component reuse between the class-based and UVM phases of the project.

5.1 Verification Plan

5.1.1 Functional Verification

Functional verification of the asynchronous FIFO ensures correct data transfer, pointer advancement, and flag behavior under normal and boundary conditions. All write and read operations are verified to confirm that FIFO ordering and data integrity are maintained across both 500 MHz clock domains under asymmetric idle cycles (0 write idle, 3 read idle).

The verification confirms that the full and empty flags — derived via Gray-code pointer comparison in wptr_handler and rptr_handler respectively — assert and deassert at the correct occupancy thresholds. The full_3_4 flag (wptr_handler: b_wptr − b_rptr_sync ≥ 384) and empty_1_4 flag (rptr_handler: b_wptr_sync − b_rptr ≤ 384) are also exercised across their transition points.

Special attention is given to the two-stage synchronizer (synchronizer.sv) to confirm that Gray-coded pointers are safely captured into the opposite clock domain with correct two-cycle latency. SystemVerilog Assertions (SVA) are used to enforce protocol rules throughout all simulation runs. Tests are executed in ModelSim or QuestaSim with waveform, log, and coverage outputs captured for analysis.

5.2 Edge Case Testing (Partially Implemented)

5.2.1 Test Case Scenarios

Test Full: This test fills the FIFO by writing data continuously (w_en=1, 0 idle cycles) until the occupancy reaches 512 entries. The wptr_handler full flag is monitored to verify it asserts exactly when Gray-code comparison detects the full condition. Any subsequent write attempt with w_en=1 while full=1 must be suppressed — the binary write pointer b_wptr must not increment, and fifo_mem must not update. The test then confirms deassertion of full once a read reduces occupancy below the threshold.

Test Empty: The FIFO is pre-loaded with data, then read continuously (r_en=1, 3 idle cycles between reads) until all entries are consumed. The rptr_handler empty flag is monitored for correct assertion when g_wptr_sync equals g_rptr_next. Any read attempt while empty=1 must be blocked — b_rptr must not increment, and data_out must not change. The test also verifies correct deassertion of empty upon the next valid write.

Test Full Error: This test attempts to write additional data while the FIFO is already full (full=1). The expected behavior is that full remains asserted, b_wptr does not advance, and no data is written to fifo_mem. The full flag should not glitch or temporarily deassert during the illegal write attempt. This validates the write guard condition: b_wptr_next = b_wptr + (w_en & !full) in wptr_handler.sv.

Test Empty Error: This test attempts to read data while the FIFO is empty (empty=1). The expected behavior is that empty remains asserted, b_rptr does not advance, and data_out remains unchanged. This validates the read guard condition: b_rptr_next = b_rptr + (r_en & !empty) in rptr_handler.sv.

Test full_3_4 (Almost Full): Data is written until occupancy reaches 384 entries. The full_3_4 flag in wptr_handler — derived from the condition (b_wptr − b_rptr_sync) ≥ 384 — must assert at this threshold. The test then reads data back until occupancy drops below 384 and confirms correct deassertion. This threshold provides early warning to upstream logic before a full overflow condition is reached.

Test empty_1_4 (Almost Empty): Data is read until occupancy falls to 128 entries (i.e., b_wptr_sync − b_rptr ≤ 384). The empty_1_4 flag in rptr_handler must assert at this boundary. The test then writes additional data to increase occupancy above the threshold and verifies correct deassertion. This flag provides early warning to downstream logic before a complete underflow condition.

Concurrent Write-Read: Simultaneous write (0 idle cycles) and read (3 idle cycles) operations are applied to stress the FIFO across both clock domains. The scoreboard verifies that all 450 burst items are received in correct order on data_out, with no data loss, duplication, or corruption. The synchronizer pipeline latency is accounted for in the expected response window. Flag behavior under sustained concurrent access is also verified.

6. Coverage Metrics (To be Implemented)

Coverage metrics measure how thoroughly the FIFO has been verified and guide additional test development. Three primary coverage types are tracked:

Functional Coverage targets the following bins:

- Flag states: all 16 combinations of {full, empty, full_3_4, empty_1_4}.
- Occupancy ranges: 0 (empty), 1–127 (low), 128 (empty_1_4 boundary), 128–383 (mid), 384 (full_3_4 boundary), 384–511 (high), 512 (full).
- Operation types: write-only, read-only, simultaneous write+read, illegal write-when-full, illegal read-when-empty.
- Flag transitions: empty to not-empty, full to not-full, empty_1_4 assertion/deassertion, full_3_4 assertion/deassertion.

Code Coverage confirms that all lines, branches, and conditions in all five RTL files are exercised:

- fifo_mem.sv: both always_ff blocks (write path and read path) fully covered.
- wptr_handler.sv: both wfull and wfull_3_4 assign expressions evaluated true and false.
- rptr_handler.sv: rempty and rempty14 assign expressions evaluated true and false.
- synchronizer.sv: two-stage pipeline for both Gray and binary pointers fully toggled.

Toggle coverage is additionally tracked to ensure all key interface and internal signals transition between 0 and 1 during simulation runs.

7. Verification Environment

The verification environment is developed in two parallel phases — a class-based environment for early functional testing and a UVM-based environment for structured, scalable verification. Both environments interface with the same DUT (asynchronous_fifo / hw_top.sv) through a SystemVerilog interface. The class-based environment was developed in earlier milestones. The UVM-based environment has been fully implemented in MS4. SystemVerilog Assertions (SVA) are planned for MS5.

7.1 Class-Based Testbench Architecture

Interface:

A SystemVerilog interface bundles all DUT ports — wclk, wrst_n, w_en, data_in, rclk, rrst_n, r_en, data_out, full, empty, full_3_4, empty_1_4 — into a single access point. A virtual interface connects the dynamic class-based testbench to the static synthesizable DUT.

Transaction Class:

Encapsulates a single FIFO operation (write or read), including randomizable fields for data_in, w_en, r_en, and timing control. The rand qualifier is applied to data_in for constrained-random stimulus, while operation type constraints ensure both write and read scenarios are covered proportionally.

Mailbox (Class-Based):

Mailboxes transfer transaction objects synchronously between the Generator and Driver, and between the Monitor and Scoreboard, maintaining single-cycle alignment with wclk and rclk respectively.

Generator:

Generates directed and random stimulus covering: normal write/read, burst write (450 items), FIFO-full scenario, FIFO-empty scenario, full_3_4 threshold crossings, empty_1_4 threshold crossings, illegal write-when-full, and illegal read-when-empty. Timing respects the 0 write idle cycle and 4 read idle cycle specification.

Driver:

Translates sequence items into pin-level wclk-domain write stimulus and rclk-domain read stimulus. The driver enforces the write guard condition (w_en=1 only when !full) and the read guard condition (r_en=1 only when !empty) matching the RTL implementation.

Monitor:

Two monitors observe the DUT interfaces independently — a write-side monitor captures w_en, data_in, and full on wclk edges; a read-side monitor captures r_en, data_out, empty, full_3_4, and empty_1_4 on rclk edges. Collected transactions are forwarded to the Scoreboard and Coverage Collector via TLM analysis ports.

Scoreboard:

A SystemVerilog queue serves as the reference model. When the write monitor observes a valid write (w_en=1, !full), the data is pushed into the queue. When the read monitor observes a valid read (r_en=1, !empty), the scoreboard pops the front of the queue and compares it against data_out. Flag correctness including full_3_4 (queue size ≥ 384) and empty_1_4 (queue size ≤ 128) is checked cycle-by-cycle.

Coverage Collector:

A dedicated subscriber collects functional coverage. Covergroups are defined for flag combinations, occupancy bins, and operation type cross-coverage as outlined in Section 6.

Environment:

Instantiates all agents, scoreboards, and coverage collectors. Provides the top-level start_of_simulation, run, and extract phases to coordinate all testbench components.

Test:

Configures the environment, selects and starts the appropriate sequence, and sets simulation parameters. Enables easy extension for new test scenarios without modifying the environment.

Testbench Top:

Instantiates the DUT (asynchronous_fifo), the SystemVerilog interface, and the test. Generates wclk and rclk at 500 MHz (2 ns period). Registers the virtual interface and initializes the simulation.

7.2 UVM-Based Testbench Architecture

The UVM-based environment uses a two-level structure: an outer environment (fifo_tb) holds the scoreboard and an inner environment (fifo_env), which contains the agent and coverage subscriber. The agent encapsulates the sequencer, driver, and two monitors — one per clock domain.

tb_top.sv — Testbench Top Module:

- Instantiates the DUT (asynchronous_fifo) and the SystemVerilog interface (fifo_if).
- Generates wclk and rclk at 500 MHz (2 ns period). Applies staggered reset: wrst_n deasserted after 10 write clock cycles, rrst_n after an additional 20 read clock cycles.
- Registers the virtual interface in uvm_config_db and launches the simulation via run_test("test2").

fifo_pkg.sv — UVM Package:

- Imports uvm_pkg and includes all testbench class files in correct compile order.
- Order: fifo_packet, fifo_sequencer, fifo_seqs, fifo_driver, fifo_monitor, fifo_monitor_in, fifo_subscriber, fifo_scoreboard, fifo_agent, fifo_env.

fifo_if.sv — SystemVerilog Interface:

- Parameterized with DEPTH=512 and DATA_WIDTH=16.
- Bundles all DUT ports: wclk, rclk, wrst_n, rrst_n, w_en, r_en, data_in [15:0], data_out [15:0], full, empty, full_3_4, empty_1_4.
- Includes auxiliary signals monstart and drvstart for transaction recording synchronization.

fifo_packet.sv — UVM Sequence Item:

- Defines a framed packet transaction: ID, LENGTH, PAYLOAD[0..N-1], PARITY transmitted in order across the FIFO.
- id is randc (no consecutive repetition); length constrained > 0; payload size constrained to equal length.
- Parity computed automatically as XOR of all payload words in post_randomize().
- write_idle_cycles and read_idle_cycles constrained to 0 and 4 respectively; registered as UVM_NOCOMPARE to exclude from scoreboard comparison.
- w_en, r_en, full, empty fields are populated by the monitor for coverage sampling.

fifo_sequencer.sv — UVM Sequencer:

- Standard UVM sequencer parameterized on fifo_packet.
- Provides the TLM seq_item_export for driver connection. No custom arbitration logic.

fifo_seqs.sv — UVM Sequences:

- fifo_base_seq: base class that raises a UVM phase objection in pre_body() and drops it in post_body() using the starting_phase handle.
- fifo_5_packets: extends fifo_base_seq; runs 100 iterations of randomized fifo_packet objects each constrained to length == 450, representing the 450-item burst specification.
- Directed sequences for edge case testing are planned for future milestones.

fifo_driver.sv — UVM Driver:

- Active driver generating write-side stimulus on the wclk domain.
- run_phase forks get_and_drive() and reset_signals() in parallel.
- send_to_dut() drives the full frame — id, length, payload[], parity — each word gated on negedge wclk iff !full. w_en is asserted for the duration and deasserted after the frame.
- On negedge wrst_n: drives data_in to high-impedance, clears w_en, and disables send_to_dut. On negedge rrst_n: clears r_en.

fifo_monitor_in.sv — Write-Side Monitor:

- Observes the wclk domain after posedge wrst_n.
- Reconstructs packet frames by sampling data_in on posedge wclk whenever w_en && !full.
- Captures id, length, payload[], parity in frame order and publishes completed packets to the scoreboard via the fifo_in analysis port.

fifo_monitor.sv — Read-Side Monitor:

- Observes the rclk domain after posedge rrst_n; also actively drives r_en.
- Asserts r_en on negedge rclk, reads id and length on posedge rclk iff !empty.
- For each payload word: samples data_out, writes a coverage packet to fifo_cov_out, deasserts r_en, waits 3 posedge rclk iff !empty cycles, then re-asserts r_en — implementing the 3-cycle read spacing.
- Publishes completed packets to the scoreboard via the fifo_out analysis port.

fifo_scoreboard.sv — UVM Scoreboard:

- Dual analysis imp ports: fifo_in (from write-side monitor) and fifo_out (from read-side monitor).
- write_fifo_in(): clones each received packet using $cast and pushes to in_queue.
- write_fifo_out(): pops from in_queue and calls packet.compare() — reports UVM_INFO on match, UVM_ERROR on mismatch.
- check_phase(): flags a UVM_ERROR if in_queue is not empty at end of simulation.

fifo_subscriber.sv — Coverage Subscriber:

- Connected to fifo_cov_out from the read-side monitor; calls cg.sample() on every received transaction.
- Coverpoints: w_en, r_en, full, empty (individual), and wr_rd_cp cross of {w_en, r_en} with bins idle, write_only, read_only, both.
- full_3_4, empty_1_4, and occupancy range bins are planned for MS5.

fifo_env.sv — Inner UVM Environment:

- Instantiates fifo_agent and fifo_subscriber.
- Connects monitor_out.fifo_cov_out to subscriber.analysis_export in connect_phase.

fifo_tb.sv — Outer UVM Environment:

- Instantiates fifo_env and fifo_scoreboard.
- Connects monitor_out.fifo_out to scoreboard.fifo_out and monitor_in.fifo_in to scoreboard.fifo_in in connect_phase.

fifo_test_lib.sv — UVM Tests:

- base_test: creates fifo_tb, enables transaction recording, prints UVM topology at elaboration, sets 20 µs drain time.
- test2: extends base_test; configures fifo_5_packets as the default sequence on the sequencer via uvm_config_wrapper.

8. Resources Requirements

Team Members and Task Allocation

Monesh Kareti (monesh@pdx.edu): Monesh is responsible for the write-side architecture across both RTL and verification. His RTL focus includes wptr_handler.sv — implementing the binary write pointer increment, Gray-code conversion, and the full / full_3_4 flag logic (wfull_3_4: b_wptr − b_rptr_sync ≥ 384) — and the fifo_mem.sv synchronous write path with overflow protection. On the verification side, Monesh develops the UVM write-side components: fifo_driver.sv (packet framing and wclk-domain stimulus), fifo_monitor_in.sv (write-side packet reconstruction), fifo_seqs.sv and fifo_sequencer.sv (sequence generation and arbitration), and fifo_packet.sv (transaction class with constraints and parity). He also leads the Design Specification documentation.

Senkathir Mutharasu (senkmuth@pdx.edu): Senkathir is responsible for the read-side architecture across both RTL and verification. His RTL tasks include rptr_handler.sv — implementing the binary read pointer increment, Gray-code conversion, and the empty / empty_1_4 flag logic (rempty14: b_wptr_sync − b_rptr ≤ 384) — and the integration of synchronizer.sv two-flop modules (sync_wptr and sync_rptr) into hw_top.sv. On the verification side, Senkathir builds the UVM read-side and infrastructure components: fifo_monitor.sv (read-side packet reconstruction and coverage sampling), fifo_scoreboard.sv (packet-level comparison and reference model), fifo_subscriber.sv (functional coverage collection), fifo_env.sv and fifo_tb.sv (environment hierarchy and TLM connections), fifo_test_lib.sv (test classes), and tb_top.sv (static testbench top). He also leads authorship of the Verification Plan.

9. References

- Cummings, C. (2002). Simulation and Synthesis Techniques for Asynchronous FIFO Design. SNUG San Jose 2002. http://www.sunburst-design.com/papers/CummingsSNUG2002SJ_FIFO1.pdf
- Aldrich, B. (2012). The UVM Primer. Object Design Works.
- IEEE Standard for SystemVerilog — Unified Hardware Design, Specification, and Verification Language. IEEE Std 1800-2017.
- Accellera Systems Initiative. Universal Verification Methodology (UVM) 1.2 User's Guide. Accellera, 2015.
- Khandelwal, T. Asynchronous FIFO Reference Implementation. GitHub. https://github.com/teekamkhandelwal/asynchronous_fifo